

ICS Exercise Week 6

2026-04-10

Problem 1: Reasoning About Reaping

Consider the following program:

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>

int main(void) {
    int i, status;
    pid_t pid;

    for (i = 0; i < 2; i++) {
        if ((pid = fork()) == 0) {
            exit(100 + i);
        }
    }

    while ((pid = waitpid(-1, &status, 0)) > 0) {
        if (WIFEXITED(status)) {
            printf("child %d exited with %d\n", pid, WEXITSTATUS(status));
        }
    }

    return 0;
}
```

1. How many child processes are created?
 2. What exit statuses can the parent observe?
 3. Must the parent observe them in a fixed order? Explain.
 4. Explain why the loop uses `waitpid(-1, &status, 0)` instead of calling `waitpid(pid, &status, 0)` only once.
-

Problem 2: Completing a Foreground-Wait Shell Fragment

Complete the following shell fragment so that:

- The child executes the command using `execve` with `cmd` and `argv`.
- The parent waits for the child only if the job is a foreground job (`bg == 0`).
- The parent does not wait if the job is a background job (`bg == 1`), but instead prints the child's PID and the command name.
- Any fork or waitpid error is reported using `unix_error`.
- If `execve` fails, the child should print an error message and exit with status 127.

```
/* Assume cmd & argv are already prepared, `bg` is 0 for foreground and 1 for
background. */
/* Assume environ and unix_error are available. */

void run_cmd(char *cmd, char **argv, int bg) {
    pid_t pid;
    int status;

    if ((pid = fork()) < 0) {
        /* Handle fork error */
    } else if (pid == 0) {
        /* child */
    }

    /* parent */
}
```

Problem 3: `waitpid` and Interrupted System Calls

A student writes the following code to reap a child:

```
int status;
pid_t ret = waitpid(pid, &status, 0);
if (ret < 0) {
    printf("waitpid failed\n");
    exit(1);
}
```

However, the program installs a signal handler for another signal, and sometimes `waitpid` returns `-1` even though the child still exists.

1. Which `errno` value is the most likely explanation?
 2. Why can this happen?
 3. Rewrite the code so that it correctly retries when interrupted, but still reports a real error.
-

Problem 4: Signal Delivery and Process Groups

A shell creates a foreground job consisting of two processes in the same process group `5000`. The shell itself is in a different process group.

1. If the user types `ctrl-c`, which process group should receive the signal?
 2. What signal is generated?
 3. If the shell instead executes `kill(-5000, SIGTSTP)`, which processes receive the signal?
 4. Why does `kill` use a negative PID to target a process group?
-

Problem 5: Signal-Driven Redirection and Critical Sections

You are writing a system utility that prints a periodic status message `"HEARTBEAT\n"` to standard output (stdout) using `SIGALRM`.

The program has two modes:

1. Normal Mode: Messages are printed to the terminal.
2. Redirect Mode: Messages are redirected to a connected client socket.

Requirements:

1. Use `alarm(1)` to trigger the heartbeat every second.
2. You must ensure that signals are blocked during critical sections (when standard output is being redirected) to prevent the signal handler from writing to an inconsistent or closed file descriptor.
3. You can use `write(1, "HEARTBEAT\n", 10);` inside the signal handler to print the message.
4. The handler must be "safe" by preserving the global `errno`.

Complete the missing parts of the program.

Background: File Descriptor Duplication (provided, no need to implement)

The skeleton code uses two system calls that are provided for you:

- `dup(fd)` — creates a copy of file descriptor `fd` and returns a new file descriptor pointing to the same underlying file/socket. Used here to save a backup of stdout before redirecting it.
- `dup2(oldfd, newfd)` — makes `newfd` point to the same underlying file/socket as `oldfd`, closing `newfd` first if it was already open. Used here to redirect stdout (fd 1) to the client socket, and later to restore it.

You do **not** need to understand the internals of these calls. What matters for this problem is: after `dup2(connfd, 1)`, any `write(1, ...)` goes to the client; after `dup2(stdout_backup, 1)`, `write(1, ...)` goes back to the terminal.

```

#include <errno.h>
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <unistd.h>

/* Helper functions provided by the CS:APP library */
int open_listenfd(int port);
void wait_disconnect(int fd);

void handler(int sig) {
    /* (a) Complete the handler: save errno, write message, and reset alarm */
}

int main(void) {
    int listenfd, connfd;
    sigset_t mask, prev;

    /* (b) Initialize signal mask to block SIGALRM */

    /* (c) Setup handler and initial alarm */

    listenfd = open_listenfd(1234);
    while (1) {
        connfd = accept(listenfd, NULL, NULL);

        /* Enter Redirect Mode */
        /* Block signals before modifying file descriptors */
        /* (d) Start Critical Section */

        int stdout_backup = dup(1);
        dup2(connfd, 1);
        /* (e) End Critical Section */

        /* Wait for client to disconnect */
        wait_disconnect(connfd);

        /* Transition back to "Normal Mode" */
        /* Block signals again during the sensitive restoration process */
    }
}

```

```
    /* (f) Start Critical Section */

    dup2(stdout_backup, 1);
    close(connfd);
    close(stdout_backup);
    /* (g) End Critical Section */

}
return 0;
}
```